ECE 601 Capstone Course
Project Report

UNDERWATER POSITIONING SYSTEM USING EXTREMELY LOW FREQUENCY (ELF) FIELDS FOR USE IN AUTONOMOUS UNDERWATER VEHICLES (AUV)

December 14, 2023

Team Member Names
Department of Electrical and Computer Engineering
College of Engineering
University of Wisconsin-Madison

1 Bill of Materials

Refer to the shared dropbox here for the BOM. Our group spent approximately 300 dollars total in creating the 2 beacons and sensor.

2 Circuit Schematics

In the wiring of the sensor and amplifier circuit Fig. 10, we connect one terminal of each magnetometer (modelled as inductor in series with a resistor) to ground and the other to the negative input of an op amp. The op amp is powered by two 9V batteries at each the positive and negative terminals. The op amp has a feedback resistor to give gain of roughly 140 at 75Hz and 225 at 40Hz. The output of the op amp is then fed to a current limiting resistor of 1k Ohms. An isolation transformer is then connected to the output of the op amp and ground on one end and the analog inputs of the high-precision AD hat on the other. A zoomed in view of sensor 1 is given in Fig. 2. The wiring to a Teensy 4.0 is given in Fig. 3, where MPU950 is an inertial measurement unit, ADS1262 is the high-precision AD hat, and SSD1306 is an OLED display. Our photos can be found at the end of our document as well as here.

Refer to Figure 4 to view our beacon schematic. For it, we chose to use an Arduino Nano which is connected to a motor controller that turns on the motor to a desired frequency. In order to verify the real time frequency of it, we also included an IR sensor which captures the RPM of the spinning magnet and displays the frequency on our OLED display module. In order to optimize power consumption, we incorporated a main switch which is used to turn on/off the entire system.

3 Simulation and Circuit Board Design

3.1 Sensor

We chose not to use a custom PCB for the sensor as we had sporadic issues and were unable to validate functionality with enough time for a PCB to be manufactured. Instead we used a solder-able breadboard for our sensor wiring, which likely resulted in the inconsistent results we have experienced. Our simulations for our sensor noise can be found here.

3.2 Beacon

For the beacon, we chose to use a proto-board in which we could solder all of our components to due to the limited time constraint and ease of service. Refer to the following for our component selection:

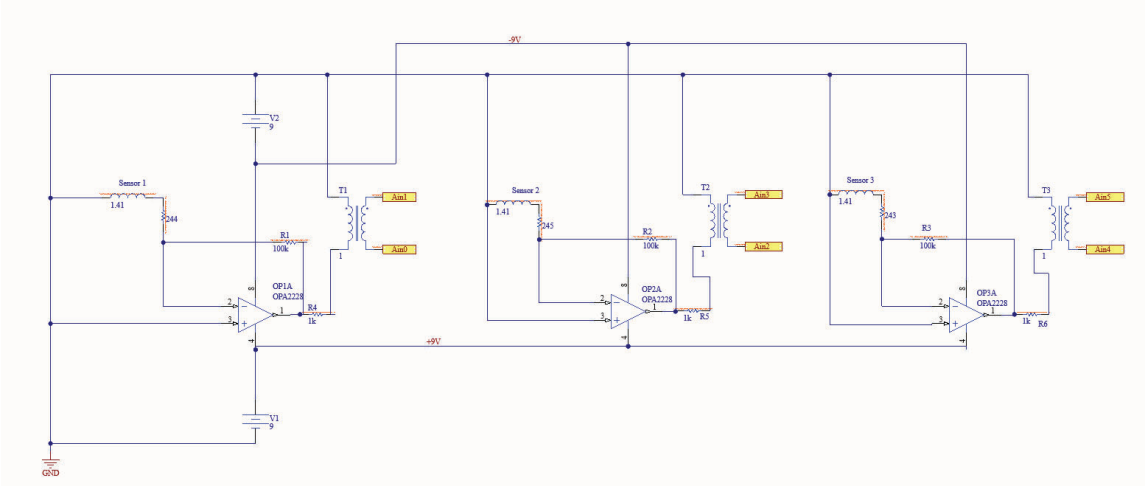

Figure 1: Sensor Connections

Component Selection

1. **Motor**
   In the process of selecting the appropriate motor for driving the magnet in our project, we initially specified Topoox 15000RPM Mini Electric Motors. However, it quickly became evident that these motors lacked the necessary torque and overall power. In an attempt to rectify this, we experimented with doubling the motors to achieve twice the power output. Despite this modification, the motors still fell short of the required power levels. Consequently, we shifted our focus and ultimately specified a 12-volt capable motor, which successfully met the torque requirements needed to drive the magnet efficiently.

2. **Motor Driver**
   After evaluating various options, we opted for the L2989 motor driver. This decision was influenced by several key features of the L2989. Firstly, the motor driver offers an optional 5V lead for VCC power, along with native regulators that can derive VCC power directly from the 12V lead used for powering the motor and magnet load. This flexibility in power options was a significant advantage.

   Furthermore, the compact size of the L2989 motor driver was a crucial factor, as it allowed for seamless integration onto our protoboard, maintaining the efficiency and neatness of our setup. In addition to these technical benefits, the cost-effectiveness of the L2989 played a role in our choice. Its affordability enabled us to purchase additional units, providing a safety net for potential replacements. This foresight was

necessary to account for the possibility of receiving faulty units from the factory or damaging them during experimentation.

3. **IR Sensor (RPM)**
   The specification of an appropriate sensor was critical for measuring the RPM of the magnet sensor effectively. After consideration, we selected an IR obstacle interruption sensor, a type of basic LiDAR sensor. This choice was driven by several key factors. Firstly, its compact size made it ideal for our application, allowing for easy integration without compromising the design. Secondly, its accuracy and ease of programming were significant advantages, ensuring reliable performance and straightforward implementation. Additionally, this sensor is both cost-effective and durable, making it a practical choice for our requirements. The sensor operates by detecting interruptions, for which we used pink tape as the interrupting medium. As the magnet spins, each full rotation results in the sensor sending a high pulse. We then applied a transfer function (frequency (HZ) * 60) to accurately determine the RPM. This approach proved to be an effective method for monitoring the rotational speed with the required precision and reliability.

4 Hardware

Our CAD files can be found here. A picture of the beacon assembly is shown in Fig. 5, and the sensor boards are shown in Fig. 8 and 9. The final sensor assembly is shown in Fig. 6 and the coil layout is shown in Fig. 7.

Refer to Fig 5 to see complete hardware assembly of beacon.

5 Coding

Below is the breakdown of the following code used for both the Sensor and Beacons.

5.1 Sensor Code

Our sensor code can be found here.

5.1.1 setup()

In the setup function, we initialize the display, calibrate the MPU, and setup data reading from the ADS1262.

5.1.2 loop()

In the main loop function, we first set the variable *dt* to the current time in microseconds for use in time updates for the lock-in amplifier. We then call the **localization()** function to calculate the sensors position in x, y and z coordinates. Next we check for if x, y or z is 20 (value to replace overflow), if so we increment the average count by the average value of each, otherwise we add x, y and z to *x_avg*, *y_avg* and *z_avg*. We repeat this function *count* times (currently set to 1000). Once we have *count* samples, we take the average of these samples for x, y and z, print to the display using the SSD1306 Arduino library [1], reset the count and avg values, then update the sensors orientation via **update_orientation()**. We end each loop of the **loop()** function by updating the time variable t.

5.1.3 readSensorData()

This is an intermediate function that was called from **loop()** instead of **localization()**. In this function **localization()** is called to calculate the sensors position, then the values of *Vtot1_1*, *Vtot2_1* and *Vtot3_1* are printed to Serial output for debugging.

5.1.4 update_orientation()

In this function we first call **mpu.update()** to read in data from the MPU9250 using the library found at [2]. We then set *g_x*, *g_y*, and *g_z* to **mpu.getAccX()**, **mpu.getAccY()**, and **mpu.getAccZ()** respectively. These values represent the component of gravity felt by each sensor axis. Next we set *phi_x*, *phi_y*, and *phi_z* to $\pi * g\_x$, $\pi * g\_y$, and $\pi * g\_z$ to approximate the angle each axis is from horizontal.

5.1.5 orientation_transformation()

In this function, we transform the values measured from our sensor so that the *Ch3* variable is always the $\hat{z}$ component by adding any vertical component of *Ch1* or *Ch2*. We also add any horizontal component of *Ch3* to *Ch2* since the azimuth angle of the sensor does not affect distance calculations.

5.1.6 lockInAmp()

This function applies a two lock in amplifiers to each channel after converting from voltage to magnetic field. The first lock in amplifier is at frequency *w1* and the second is at frequency *w2*. All intermediate variables in this function are unique to the corresponding beacon and channel, and use the naming scheme *Vy_x* where y is replaced by the number of the corresponding channel 1, 2, or 3 and x is replaced by the beacon number, 1 for *w1* and 2 for *w2*. The lock in amplifier functions by finding the in-phase and quadrature components of a given signal with respect to a reference sinusoidal wave. Next the in phase

and quadrature components are fed through a digital rc filter, and the sum of squares of these components is the final signal.

5.1.7 localization()

The localization function first calls **lockInAmp()** to calculate the the sensor signal at *w1* and *w2*. Then four intermediate values, *k1*, *b1*, *r1* and *Theta1*, are calculated via the equations in Eq. 1, which were derived from Professor Strachen based on equations used in [3] and [4]. We also calculate *k2*, *b2*, *r2*, and *Theta2* with the same equations, but replacing all X_1 components with their X_2 counterpart. In the code, several tricks are used to avoid overflow and erroneous values such as factoring out powers of 10 from squares, and taking the absolute value of expressions within square roots. In addition, since values within arcsin must be between -1 and 1, we round the value to be within his range to prevent errors using *(c/abs(c))*min(1, abs(c))* where c is the expression within arcsin.

$$
\begin{aligned}
k_1 &= \frac{8}{3} \cdot B_{\text{mag1}}^2 \cdot \left(\frac{2\pi}{\mu_0 \cdot m}\right)^2 \\
b_1 &= \left(\frac{8\pi}{\mu_0 m}\right)^2 \cdot B_{z_1}^2 \\
r_1 &= \left(\frac{\left[\sqrt{3} \cdot \sqrt{12 \cdot k_1^2 - 20 \cdot b_1 \cdot k_1 + 3 \cdot b_1^2} + 10 \cdot k_1 - 3 \cdot b_1\right]^{1/6}}{1.348 \cdot \sqrt[3]{k_1}}\right) \\
\theta_1 &= \pi/2 - 1/2 \cdot \arcsin\left(\frac{8\pi \cdot r_1^3}{\mu_0 \cdot m} \cdot B_{z_1}\right) \\
d_1 &= r_1 \cdot \sin(\theta_1) \\
d_2 &= r_2 \cdot \sin(\theta_2) \\
y &= \left(d_1^2 + d^2 - d_2\right)/(2 \cdot d) \\
x &= \sqrt{d_1^2 - y^2} \\
z &= r_1 \cdot \cos(\theta_1)
\end{aligned}
\tag{1}
$$

We end the **localization()** function with a final check for any erroneous values in x, y, and z, if an error is detected, we set the value to a maximum detectable distance of 20, however this is likely to change during our testing depending on sensitivity.

5.1.8 GetMag()

This function is used to calculate the magnetic field given a voltage, frequency and sensor. The equations used are given in Eq.2 where Z_in the the impedance of the sensor and n is

the number of turns in the sensor.

$$\begin{aligned} Z_{\text{in}} &= \sqrt{R_s^2 + (I_s \cdot w)^2} \\ \text{gain} &= \frac{R_f}{Z_{\text{in}}} \\ B &= \frac{V}{n \cdot \text{ gain } \cdot A \cdot w} \end{aligned} \tag{2}$$

5.1.9 readADC() and mapfloat()

This code was provided by Professor Strachen and is used to read the sensor data from ADS1262.

5.2 Beacon Code

Our beacon code can be found here.

5.2.1 Libraries

We were required to use SPI, Wire (I2C), and Adafruit's graphics for OLED libraries to run the code.

5.2.2 Motor Control

To initiate motor movement, we had to create 2 Digital-Out pins from the Arudino Nano that would be connected to the motor input pins on the motor controller. We also had to connect a PWM pin on the Nano to the motor enable pin on the controller so we can set desired frequencies. It is also crucial to ensure you have a large enough battery to power the motors attached to the motor controller as the Nano iteself would be insufficient.

5.2.3 IR Sensor (RPM)

The IR sensor worked by sending a voltage signal to the Nano, in which it would read its change as a digital signal. To first make this work, we had to adjust the potentiometer on the IR sensor itself which allowed us to change the sensitivity to the desired distance we wanted to place it from the spinning motor. We accomplished the accuracy of this by reading the output pin on an oscilloscope and comparing it to what the Nano was reading. Once they matched, we pushed the Nano's output to the OLED display module so a user could read the real-time frequency. Specifcally, we made it print the frequncy in Hz, RPM, and elapsed time. This allows the user to verify before testing and notice any changes if the battery was draining.

5.2.4 OLED Display Module

The display module on our device works by using the I2C communication protocol for data transfer. It uses 2 lines for communication: SDA (Serial Data Line) and SCL (Serial Clock Line). With the help of specified libraries, we were able to write code that defines screen size, initializes the display, and draw on it using I2C communication.

6 Operating Instructions

6.1 Sensor Operation

In order to operate the sensor, the user must first take the yellow top off the sensor housing, which is press-fit and will come off with relative ease. Next, the user will plug the 9V batteries into the the amplifier circuit via the connection wires. Lastly, the user should plug the Teensy 4.0 into a portable battery bank using a micro-usb cable. The OLED screen should turn on and initialize it start-up sequence. Follow the on screen instructions, and after the initial set-up, the sensor will begin reading values and displaying its location in x, y and z coordinates, assuming the 40Hz beacon is located at the origin.

6.2 Beacon Operation

To operate the beacon, the user has to remove the press-fit lid and plug in a power source for both the Arduino Nano and motor. A 9V alkaline battery will be sufficient for powering the Nano, but not the motor. In this case, the user can use a battery pack that does not get drained as quickly so the motor can maintain a constant frequency for a longer period of time. The user will then be able to flip the switch to turn on the beacon. The motor will begin to spin and the IR RPM sensor will start displaying the frequency on the display module. In order to set different frequencies, the user will have to edit and re-upload the code to the Nanos.

7 Troubleshooting

7.1 Sensor

Our sensor currently outputs reasonable values, however the lock-in amplifier appears to have a significant amount of noise at 75Hz. To fix this issue, we would need to test alternate frequencies to find one with minimal noise. We have not yet performed a full scale test with our beacons, so these values have yet to be confirmed. If the values are not consistent with what we expect, our next steps would be to double check for any errors in the code, specifically in **localization()**. If this does not solve the issue, then there is likely something incorrect in the **GetMag()** function, so we would then double check these values. If the issue persists, there is likely an issue in our circuitry, and we would have to

validate operation using an oscilloscope. With more time, we would have designed a custom PCB to reduce the length of any wires, as well as ensure a secure interface between the ADS1262 and the Teensy 4.0. Given these steps and sufficient working time, we would be able to significantly improve our sensors accuracy.

7.2 Beacon

The motors within our beacons that spin the magnets successfully work and the IR sensor can accurately obtain RPM values. However, we came across a crucial issue during the final steps. We forgot that the requirements were for the beacons to be able to spin for an hour and our 9V battery attached to the motor controller was not sufficient. To fix this, we drilled a hole through the enclosing so we would be able to have a portable charger run the motors. This did not work, so we are planning on creating a 9V battery pack by placing them all in series so the motors can maintain constant frequencies for longer periods of time.

8 Testing and Experiments

To perform initial tests on our sensor, we first set both f_1 and f_2 to 40Hz and set the distance between beacons to 1m, then ran one beacon provided by Professor Strachen at roughly 40Hz. The sensor output values that were too far away from the actual distance to be correct, but this is likely due to m_{magnet} being set to the value for our larger beacon magnet. For the next test, we set one of Professor Strachen's beacons at roughly 40Hz and the other at roughly 75Hz, and updated the corresponding values in our code. These results were significantly better, but still not completely accurate, and the z-axis seemed to read the correct value. Our beacons are not currently fully assembled, but we will do a more comprehensive test on Friday.

9 Future Work

9.1 Sensor

In the future, more testing needs to be done to validate the calculated distances. We also would like to run more comprehensive tests to determine which frequencies to use in order to minimize external noise. Lastly, we would like to clean up our Arduino code to reduce clutter and keep a consistent naming scheme throughout, as well as add comments that better explain the working features for anyone that decides to work on our sensor in the future.

9.2 Beacon

The project has yielded several key lessons that emphasize both efficiency and functionality in design and maintenance. Firstly, increasing the wire size is essential for better serviceability, aligning with the standard practice in electrical enclosures of wrapping the service wire around the perimeter at least twice for stability. Secondly, it's beneficial to print sample sections of the design for a dry fit before manufacturing the entire parts, a strategy that saves both filament and time. Thirdly, using acrylic drill bits is recommended when reworking PLA to minimize the risk of plastic cracking, with an optimal drilling speed of 400-600 RPM for best results.

Additionally, standardizing wire colors across both beacons simplifies maintenance, with a recommendation to use intuitive color coding, such as red for power and black for ground. Further enhancing the beacon's functionality, specifying a battery with a higher ampere-hour rating is advised to ensure longer operational time. Lastly, integrating a potentiometer to adjust the beacon from the instrument cluster adds a layer of user control and flexibility, allowing for easy adjustments of the beacon's settings.

References

[1] F. Durigon, "Driver for oled displays with ssd1306 or sh1106," https://github.com/durydevelop/arduino-lib-oled.

[2] Hideakitai, "Mpu9250," https://github.com/hideakitai/MPU9250, arduino library for MPU9250 Nine-Axis (Gyro + Accelerometer + Compass) MEMS MotionTrackingâ¢ Device. This library is based on the great work by kriswiner, and re-writen for the simple usage.

[3] M. Manteghi, "A navigation and positioning system for unmanned underwater vehicles based on a mechanical antenna," 2017, pp. 1997–1998.

[4] A. Sheinker, B. Ginzburg, N. Salomonski, L. Frumkis, and B. Z. Kaplan, "Localization in 3-d using beacons of low frequency magnetic field," *IEEE Transactions on Instrumentation and Measurement*, vol. 62, pp. 3194–3201, 2013.

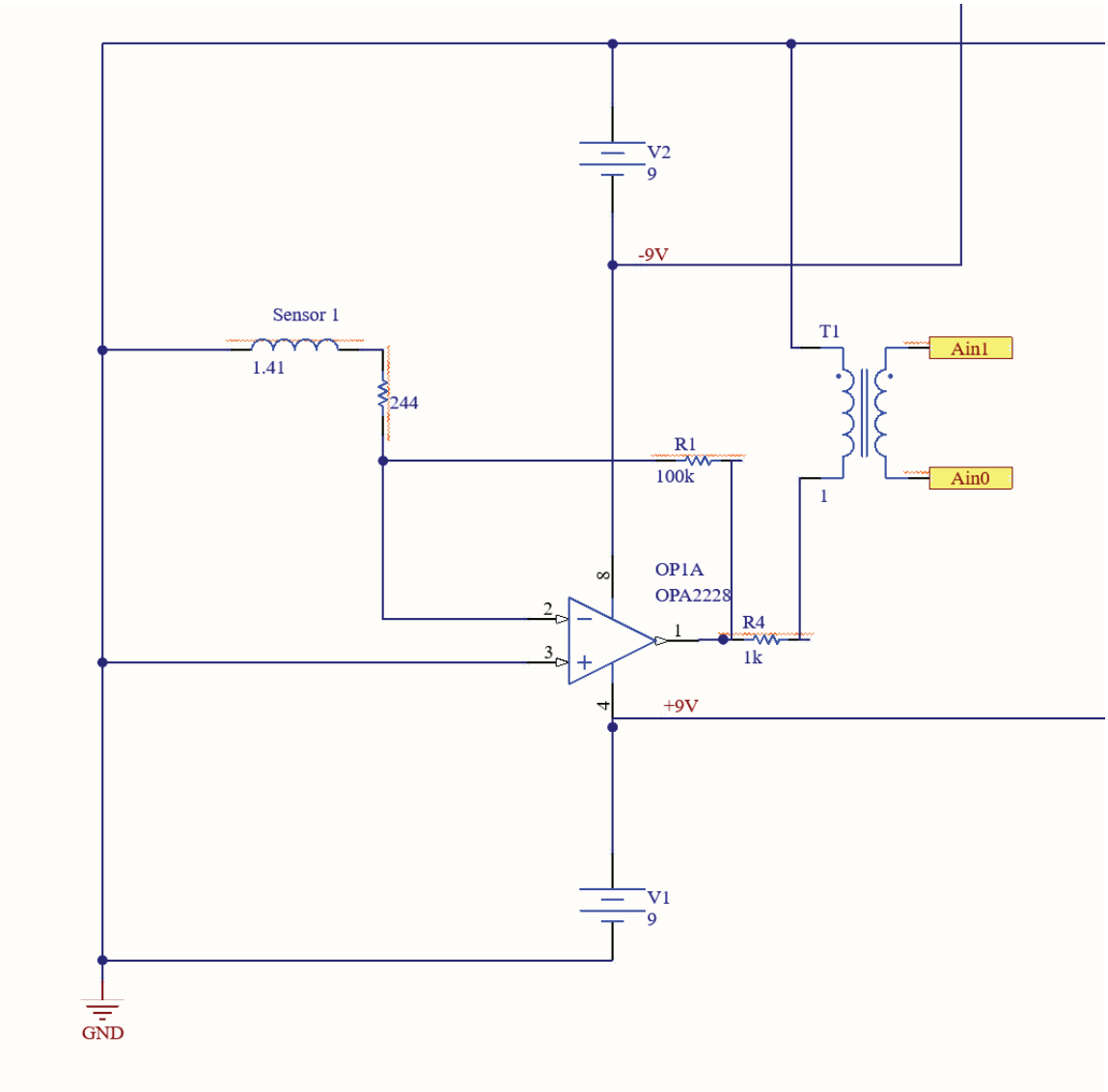

Figure 2: Sensor Connections Zoomed

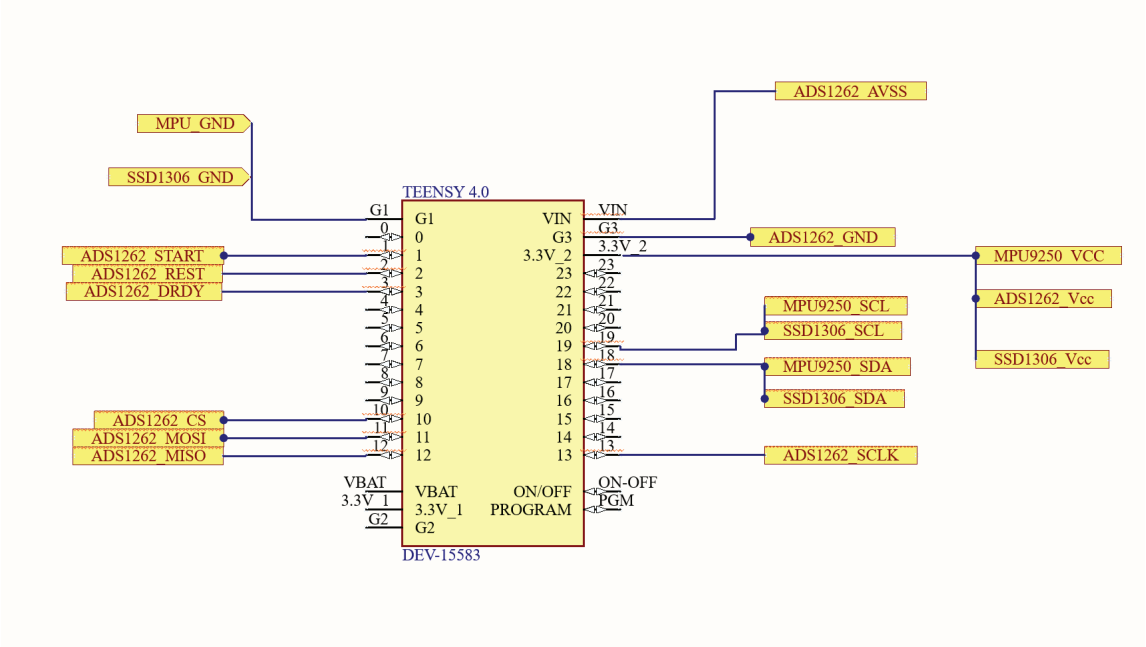

Figure 3: Teensy 4.0 connections

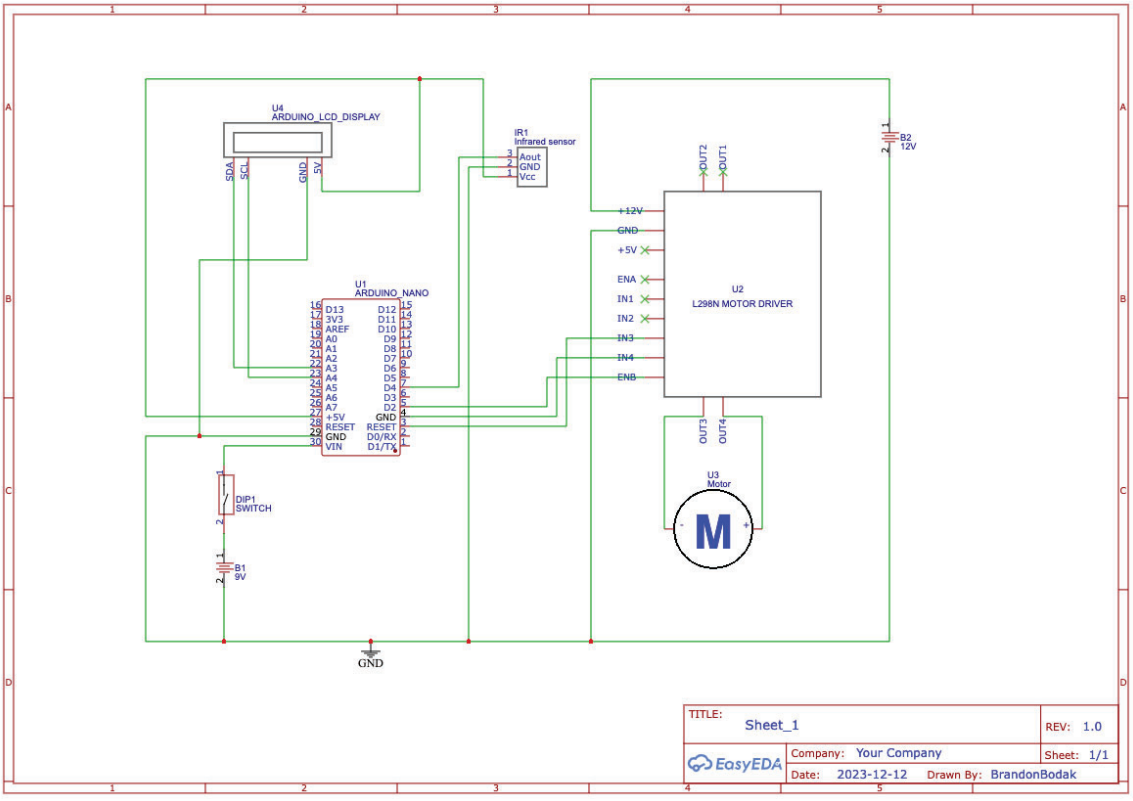
U4
ARDUINO_LCD_DISPLAY
SDA
SCL
GND
5V
IR1
Infrared sensor
Aout
GND
Vcc
U1
ARDUINO_NANO
D13
3V3
AREF
A0
A1
A2
A3
A4
A5
A6
A7
+5V
RESET
GND
VIN
D12
D11
D10
D9
D8
D7
D6
D5
D4
D3
D2
GND
RESET
D0/RX
D1/TX
DIP1
SWITCH
B1
9V
GND
OUT2
OUT1
+12V
GND
+5V
ENA
IN1
IN2
IN3
IN4
ENB
U2
L298N MOTOR DRIVER
OUT3
OUT4
U3
Motor
M
B2
12V
TITLE:
Sheet_1
REV: 1.0
EasyEDA
Company: Your Company
Sheet: 1/1
Date: 2023-12-12
Drawn By: BrandonBodak

Figure 4:

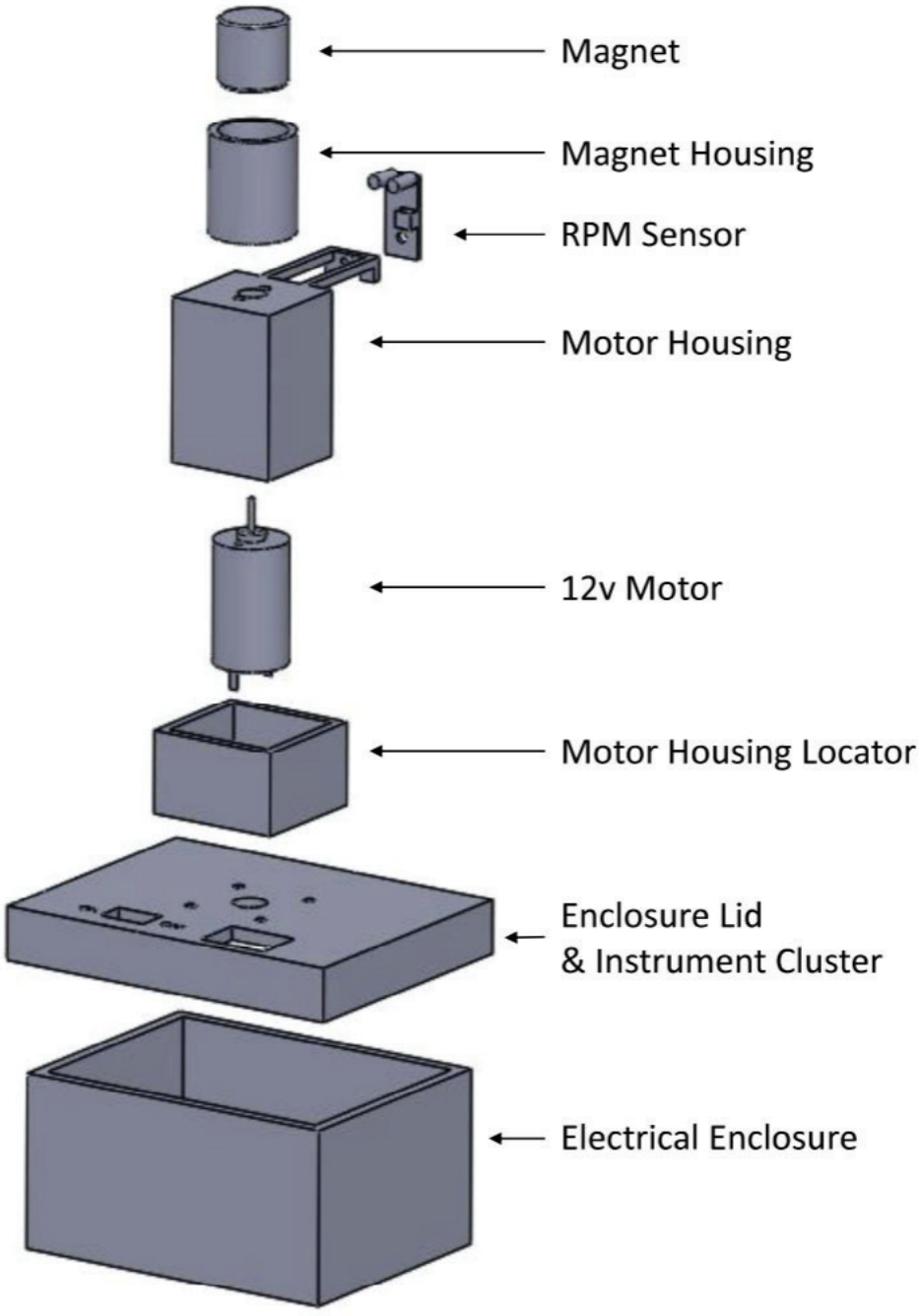

Figure 5: Beacon Exploded Assembly

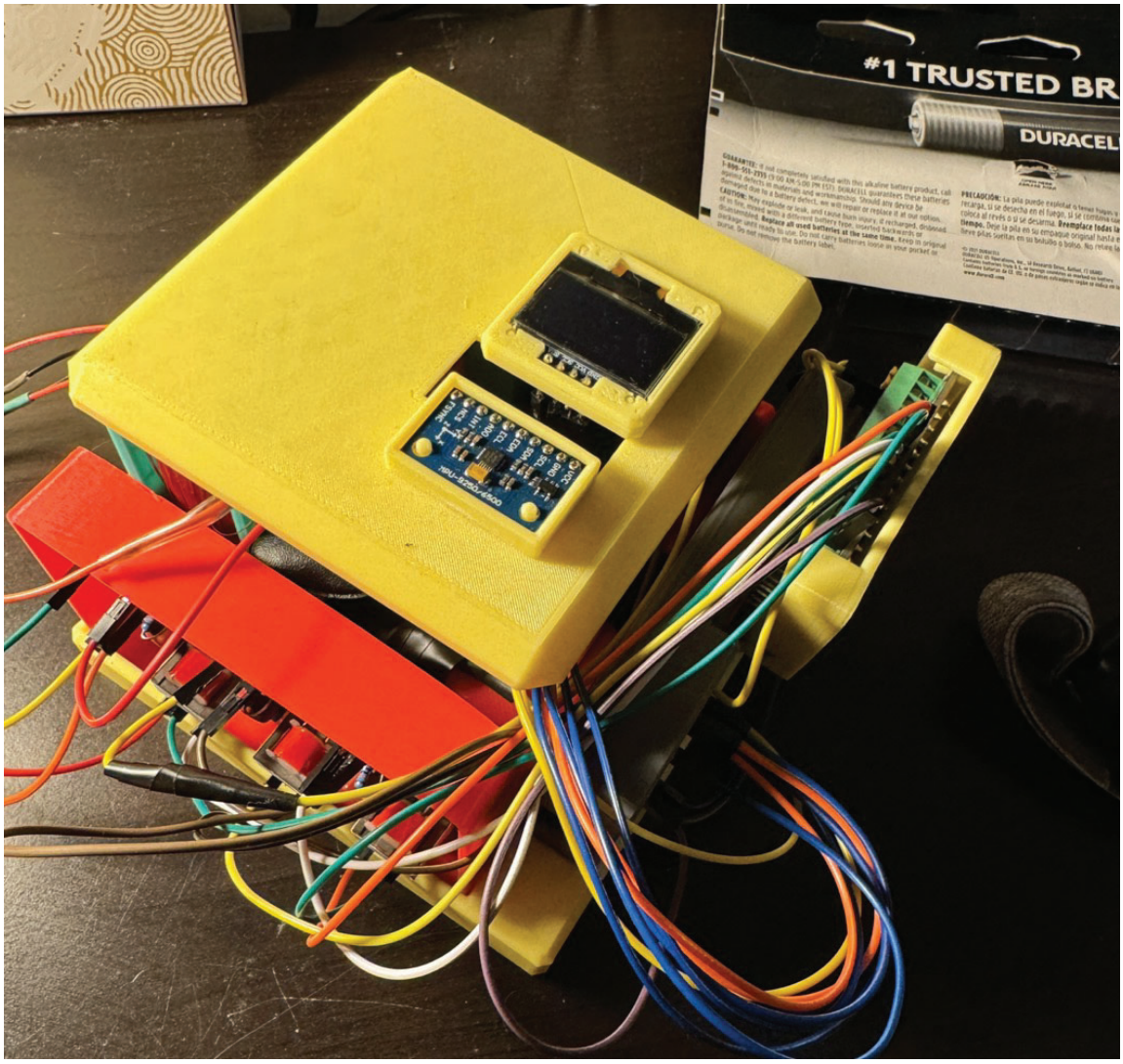

Figure 6: Sensor Full Assembly

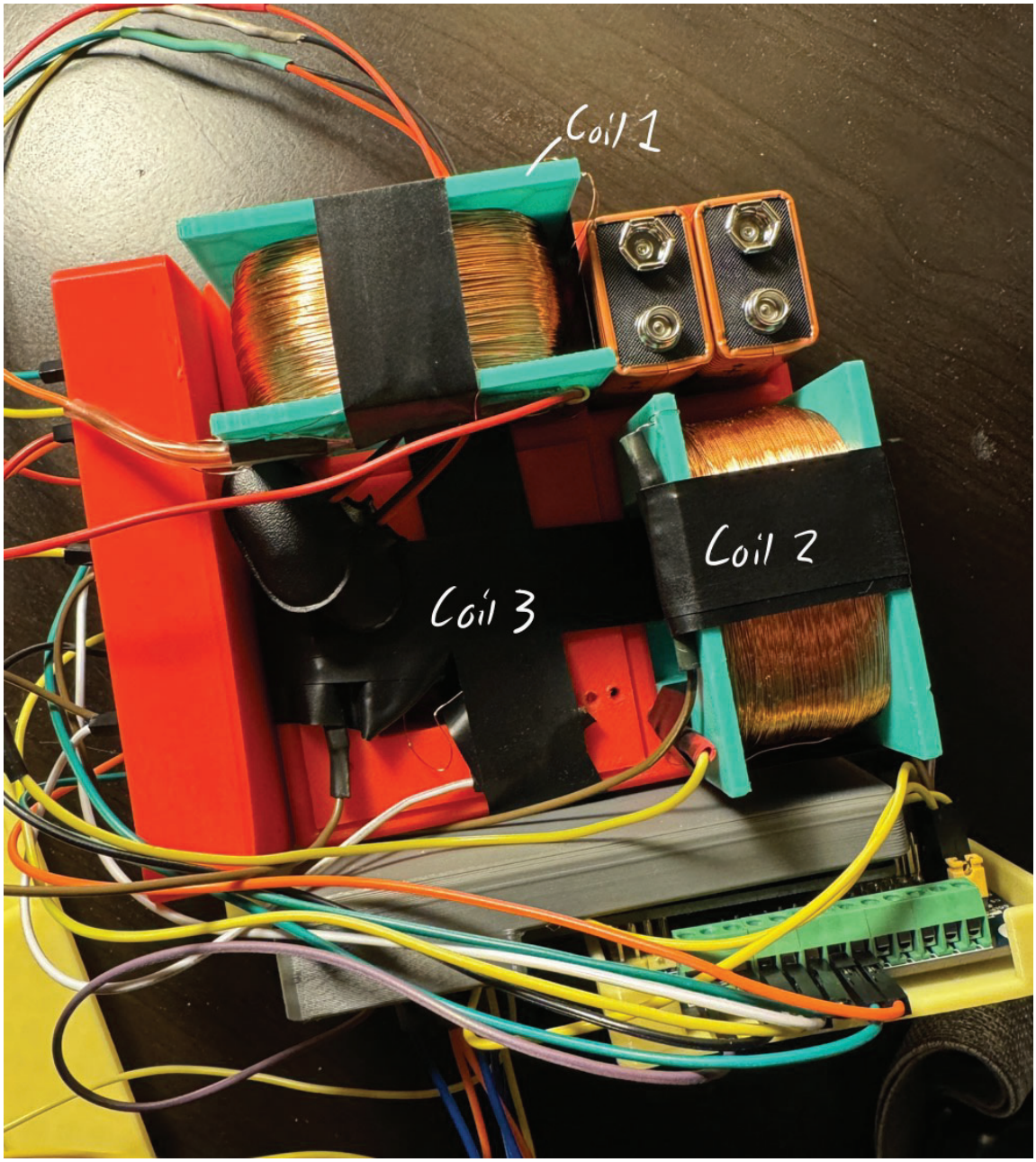

Figure 7: Sensor Coil Layout

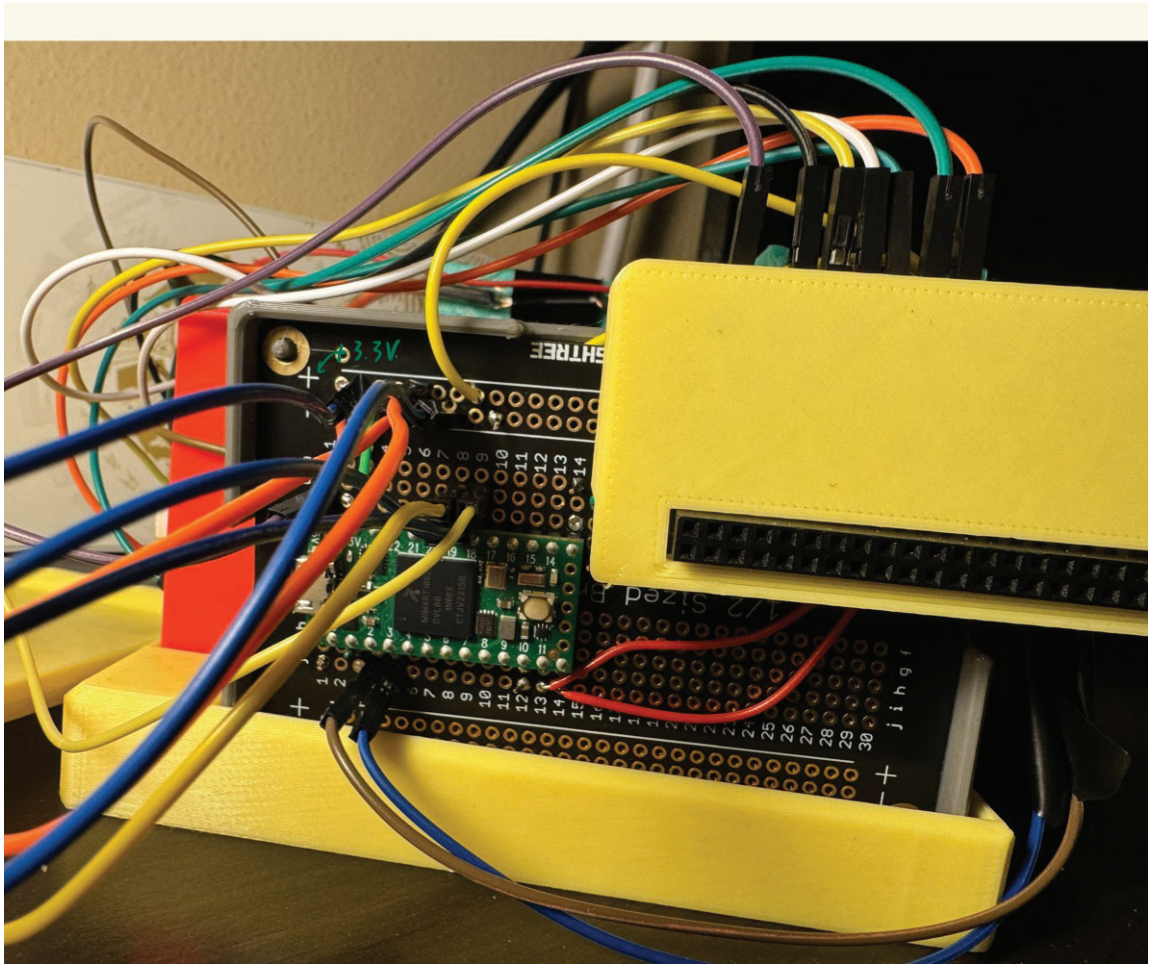

Figure 8: Sensor Main Board

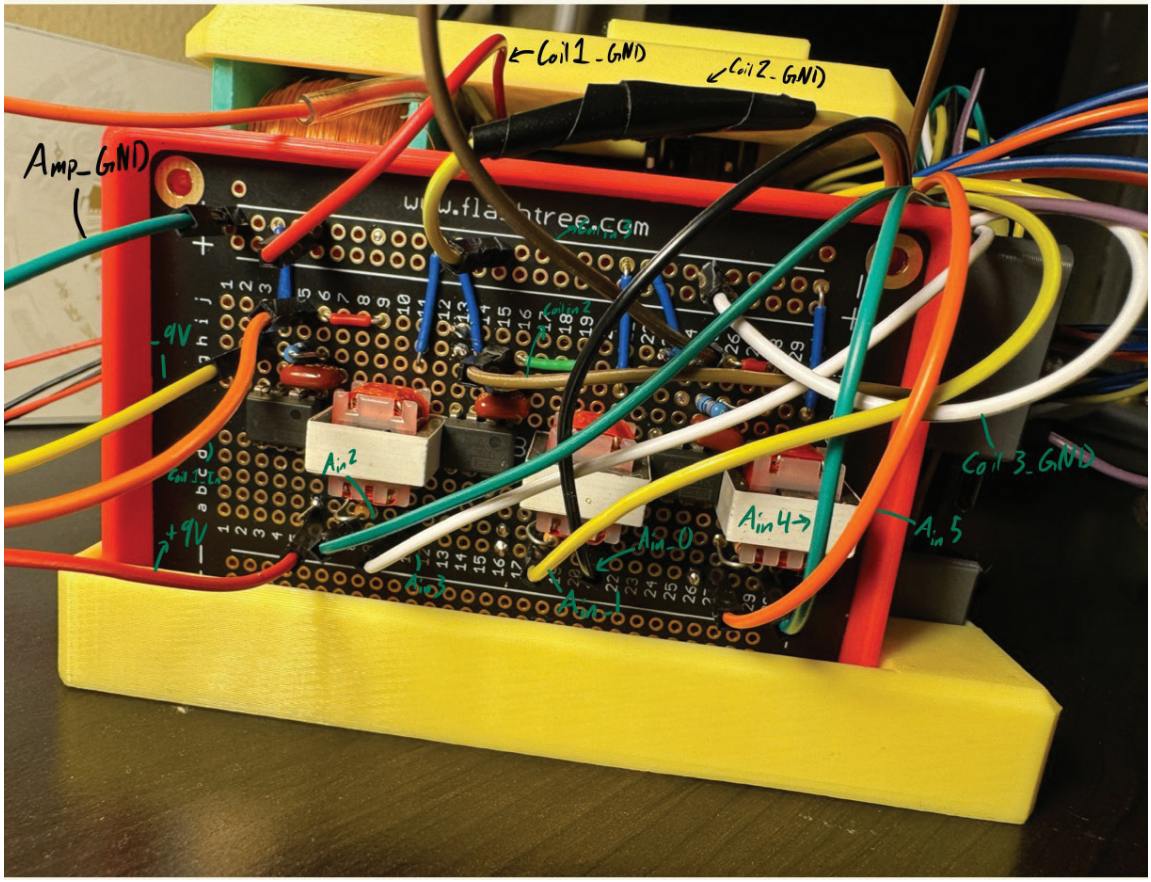

Figure 9: Sensor Amplifier Board

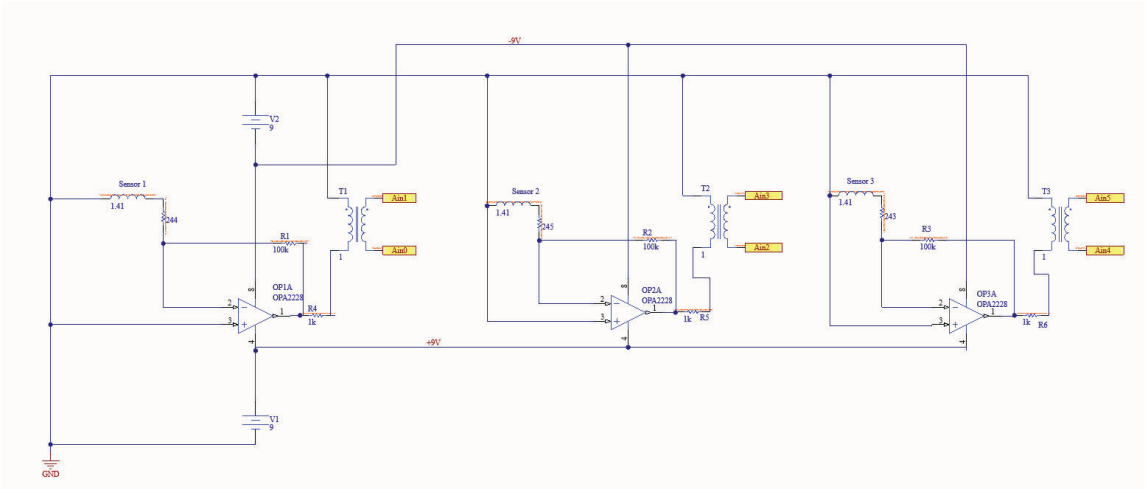

Figure 10: Sensor Connections